

## EXP16: Feed Forward Neural Network

**Name:** Y. Roshitha (192424400)

**Aim:**

To develop a Python program that implements a Feed Forward Neural Network with one hidden layer, trained using backpropagation, so that it correctly learns the XOR logic function.

**Algorithm:**

- Step 1: Read the input dataset (features) and the corresponding target output values.
- Step 2: Initialize the weights and biases for the input-to-hidden and hidden-to-output connections with small random values.
- Step 3: Perform Forward Propagation: compute the hidden layer activations and then the final output activation using the sigmoid function.
- Step 4: Compute the error as the difference between the actual output and the predicted output.
- Step 5: Perform Backward Propagation: compute the gradients at the output layer and hidden layer using the derivative of the sigmoid function.
- Step 6: Update the weights and biases using the computed gradients and a chosen learning rate.
- Step 7: Repeat Steps 3 to 6 for a fixed number of epochs, periodically displaying the loss.
- Step 8: After training, display the final predicted output for every input pattern.

**Source Code:**

```
# Program to implement a Feed Forward Neural Network

import numpy as np

np.random.seed(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# XOR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

weights_input_hidden = np.random.uniform(size=(input_neurons, hidden_neurons))
```

```

weights_hidden_output = np.random.uniform(size=(hidden_neurons, output_neurons))
bias_hidden = np.random.uniform(size=(1, hidden_neurons))
bias_output = np.random.uniform(size=(1, output_neurons))

learning_rate = 0.5
epochs = 10000

for epoch in range(epochs):
    # Forward Propagation
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    final_output = sigmoid(final_input)

    # Backward Propagation
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(weights_hidden_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

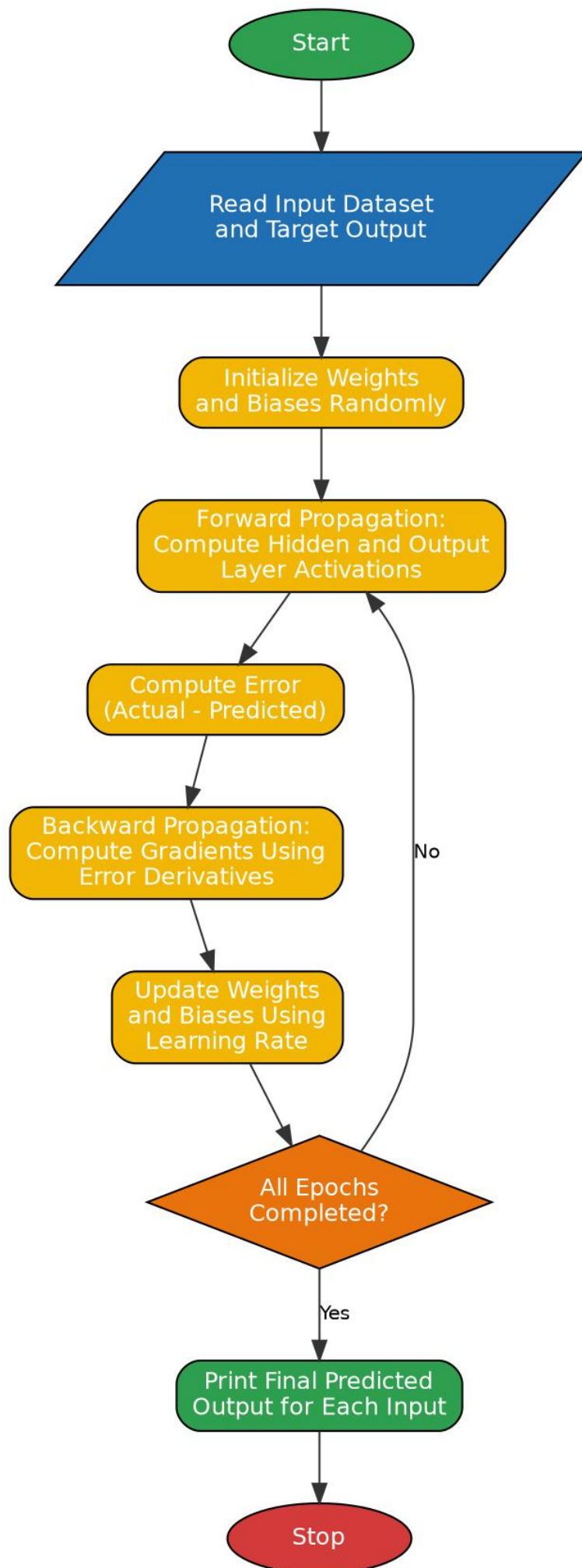
    # Update Weights and Biases
    weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate
    weights_input_hidden += X.T.dot(d_hidden) * learning_rate
    bias_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    if (epoch + 1) % 2000 == 0:
        loss = np.mean(np.square(error))
        print(f'Epoch {epoch + 1}, Loss: {loss:.6f}')

print("\nFinal Predicted Output (rounded):")
predicted = sigmoid(np.dot(sigmoid(np.dot(X, weights_input_hidden) + bias_hidden),
weights_hidden_output) + bias_output)
for i in range(len(X)):
    print(f'Input: {X[i]} -> Predicted: {round(predicted[i][0], 4)} -> Class:
    {int(round(predicted[i][0]))}')

```

**Flowchart:**



## Output:

```
>>> ===== RESTART: C:/Users/yanar/Documents/CSA1717/EXP16.py =====
Epoch 2000, Loss: 0.004116
Epoch 4000, Loss: 0.000914
Epoch 6000, Loss: 0.000488
Epoch 8000, Loss: 0.000328
Epoch 10000, Loss: 0.000246

Final Predicted Output (rounded):
Input: [0 0] -> Predicted: 0.0171 -> Class: 0
Input: [0 1] -> Predicted: 0.9869 -> Class: 1
Input: [1 0] -> Predicted: 0.9829 -> Class: 1
Input: [1 1] -> Predicted: 0.0151 -> Class: 0
>>> |
```

## Result:

Thus the program to implement a Feed Forward Neural Network was written, executed successfully and the output was verified.